

The background of the slide is a photograph of a vast, arid landscape. In the foreground, there are rolling hills with a textured, brownish surface, possibly covered in low-lying vegetation or sand. The hills recede into the distance, creating a sense of depth. In the far background, a prominent, conical mountain peak rises above the horizon, its silhouette softened by a light haze. The sky is a pale, uniform color, suggesting a clear or slightly overcast day. The overall lighting is soft and even, with no harsh shadows, giving the scene a serene and expansive feel.

Lecture 6

Transformers: Attention Mechanism

Advanced Topics in Artificial Intelligence · UFABC

Lecture outline

Part 1 — Motivation Polysemy, ATIS task, and requirements for a solution

Part 2 — Self-Attention Contextual representation, dot product, softmax

Part 3 — Multi-Head Attention Parallel heads, concatenation and projection

Part 4 — Positional Encoding Permutation invariance problem, positional embeddings

Part 5 — Full Encoder Q/K/V, residual connections, LayerNorm, complete block

Part 6 — Applications Slot filling, task types, PyTorch code

Part 1 — Motivation

From RNNs to Transformers

In the previous lecture we covered RNNs and LSTMs as sequence models. Why aren't they enough for modern NLP?

Sequential processing

RNNs compute h_t from h_{t-1} — impossible to parallelize. N tokens = N serial steps, even on GPUs.

❌ Slow training on long sequences

Vanishing gradient

Gradients dissipate when backpropagating through many steps. LSTMs mitigate but do not eliminate the problem.

❌ Long-range dependencies are hard to learn

Compressed context

The entire sequence history is compressed into a single vector h_t . Information from distant tokens progressively dilutes.

❌ In long sequences, h_t carries almost no information from the first tokens

Transformer (Vaswani et al., 2017) eliminates recurrence: each token connects directly to all others via **self-attention** — fully parallel processing, global context, no vanishing gradient in attention.

The polysemy problem

Polysemy: the same word has different meanings depending on context.

"station"

- *"I arrived at the train **station**."* → railway station
- *"She works at a radio **station**."* → broadcasting station

"bank"

- *"I deposited money at the **bank**."* → financial institution
- *"We sat on the river **bank**."* → riverbank

The problem with static word embeddings

✗ Word2Vec and GloVe assign **one single vector** per word — regardless of context.

"bank" always has the same embedding, blending both meanings together.

✓ We need **contextual representations**: the embedding of "station" should depend on surrounding words.

"train station" → close to railway-related terms

"radio station" → close to media-related terms

ATIS task — Slot Filling

ATIS (Airline Travel Information System): classify each word in a sentence into semantic categories (*slots*).

```
Sentence: I    want  to   fly  from  boston  at    7   am   and  arrive  in   denver  at   11   in   the  morning
Label:     0     0    0    0    0    B-from  0    B-dep  I-dep  0    0    0    B-arr   0    B-arr  I-arr  I-arr  I-arr
```

B-fromloc.city_name "boston" = origin city

B = beginning of the entity

B-depart_time.time "7 am" = departure time

I = continuation of the entity

B-toloc.city_name "denver" = destination city

O = outside any entity

Challenge: the label of each word depends on context. "at" before "boston" is different from "at" before "7 am". The model needs to look at the surroundings of each token to assign the correct label.

Three requirements for a solution

1. Output same length as input

For slot filling, each input token needs a corresponding label.

Input: N tokens → Output: N labels

2. Capture surrounding context

The representation of each word must take all other words in the sequence into account.

"station" in "train station" ≠ "station" in "radio station"

3. Capture word order

"cat ate mouse" ≠ "mouse ate cat"

The position of each token carries essential semantic information.

RNNs partially meet these — they process tokens in order and maintain a hidden state. But future context is ignored (unless bidirectional) and long-range dependencies suffer from vanishing gradient.

The Transformer solves all three requirements in parallel and efficiently via **self-attention**.

Part 2 — Self-Attention

Intuition: contextual representation

Core idea: the new representation of a word is a **weighted average** of the embeddings of all other words.

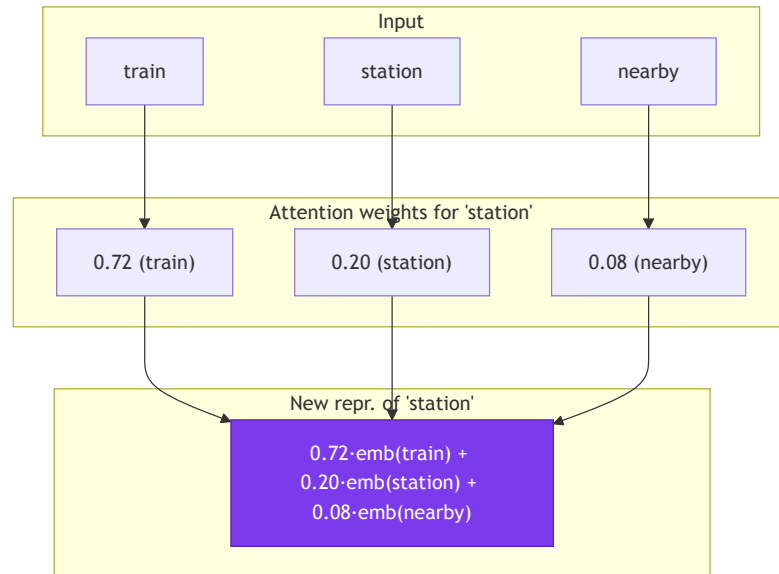
The weight of each word is **proportional to its similarity** to the target word.

Example: "station" in "train station"

- High similarity with "train" → high weight
- Low similarity with "the", "at" → low weight
- New representation of "station" moves toward "train" in vector space

Before (static embedding): "station" = fixed point in space

After (self-attention): "station" = weighted combination of its context



How to compute similarity

Similarity between word i and word j = $\mathbf{w}_i \cdot \mathbf{w}_j$

Similarity = dot product of embeddings

To compute the weight that word j receives when representing word i, we use the dot product of their embeddings.

Why dot product?

Vectors pointing in the same direction have a high dot product; orthogonal vectors have a dot product of zero. The dot product measures the **semantic alignment** between embeddings — words that co-occur in similar contexts will have aligned embeddings after training. The full formula (with softmax and scaling) comes in the next slides.

Why does the dot product capture similarity?

$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos \theta$ — measures the **directional alignment** between two vectors

$\theta \approx 0^\circ \rightarrow$ **high dot product**

"cat" and "kitten" share contexts $\rightarrow$
embeddings point in the same direction $\rightarrow$
high similarity

$\theta = 90^\circ \rightarrow$ **zero**

"cat" and "quantum" rarely co-occur $\rightarrow$
orthogonal embeddings $\rightarrow$ unrelated

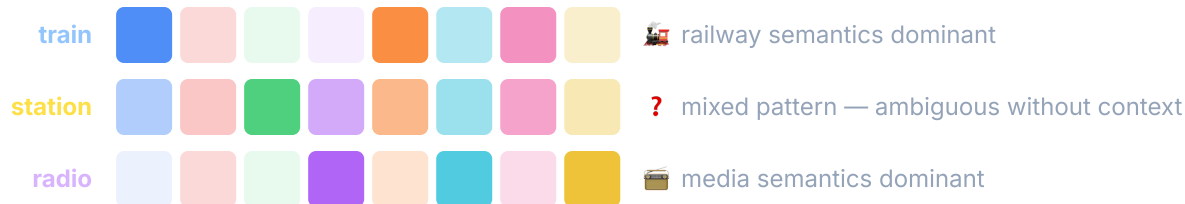
$\theta = 180^\circ \rightarrow$ **negative**

Antonyms may have embeddings pointing in
opposite directions

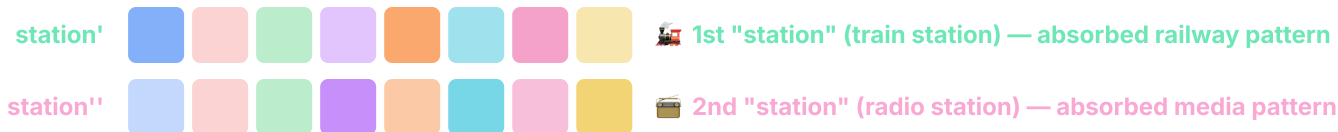
Why do embeddings align? During training, words that co-occur in similar contexts receive similar gradients $\rightarrow$ their vectors converge toward the same direction in d-dimensional space. The dot product is **efficient** ($O(d)$, differentiable) and captures exactly this alignment.

Self-Attention: embedding as a "color fingerprint"

Sentence: "train station nearby the radio station" — same word, two meanings



Similarity (dot product): words with similar patterns receive higher weight — the neighboring context determines which meaning of "station" prevails



Self-Attention Formula

$$\text{Attention}(\mathbf{W}) = \text{softmax}\left(\frac{\mathbf{W}\mathbf{W}^\top}{\sqrt{d}}\right) \mathbf{W}$$

Step-by-step interpretation

- $\mathbf{W}$ ($N \times d$): embedding matrix of the N words
- $\mathbf{W} \cdot \mathbf{W}^\top$ ($N \times N$): all pairwise dot products between words
- Divide by $\sqrt{d}$: stabilizes gradients (prevents saturated softmax)
- Row-wise softmax: attention weights sum to 1 per word
- Multiply by $\mathbf{W}$: combine embeddings with the weights

Why divide by $\sqrt{d}$?

With high-dimensional embeddings d , dot products have large magnitude — this saturates the softmax (gradients ≈ 0). Scaling by $1/\sqrt{d}$ keeps the variance of scores around 1, regardless of d .

Computational efficiency: the entire operation — N^2 dot products, scaling, softmax, and weighted combination — is expressed as dense matrix multiplications. GPUs execute this in parallel for all words simultaneously, with no explicit loops.

Result: a new $N \times d$ matrix where each row is a contextualized representation of the corresponding word — incorporating information from all other words in the sequence.

Example: Self-Attention Step by Step (d=2)

"train station nearby the radio station" — "station" appears twice with identical initial embeddings

Initial embeddings W (4×2)

	d_1	d_2
tr.	0.9	0.1
sta ₁	0.6	0.6
ra.	0.1	0.9
sta ₂	0.6	0.6

sta₁ and sta₂ have **identical** embedding 0.6, 0.6

Attention: $\text{softmax}(W \cdot W^T \div \sqrt{2})$

	tr.	sta ₁	ra.	sta ₂
tr.	0.3	0.25	0.19	0.25
sta ₁	0.24	0.26	0.24	0.26
ra.	0.19	0.25	0.3	0.25
sta ₂	0.24	0.26	0.24	0.26

 rows sta₁ = sta₂: identical attention weights → same output representation!

Part 3 — Positional Encoding

The problem: no position, no order

Self-attention is permutation-invariant:

If we shuffle the word order, the weights change but the operation cannot distinguish "position 1" from "position 5".

✗ Without modification, the Transformer treats as equivalent:

"cat ate mouse" = "mouse ate cat" = "ate mouse cat"

The model completely loses word order information.

Why does this matter?

Subject vs. object — "John saw Mary" ≠ "Mary saw John": position determines who acts.

Modifiers — "big angry dog" ≠ "angry big dog": adjective order affects grammatical construction.

Slot filling — in "fly from boston at 7am", the position of "boston" after "from" defines it as origin city.

Solution: Positional Encoding

Idea: add a positional vector to each word's embedding before it enters the Transformer.

$$\mathbf{x}'_i = \mathbf{x}_i + \mathbf{p}_i$$

Numerical example:

Token	Embedding	+ Pos. Enc.	= Input
"cat"	(0.5, 7.1)	+ (1.3, 3.9)	= (1.8, 11.0)
"sat"	(1.2, 5.3)	+ (6.3, 3.7)	= (7.5, 9.0)

The final inputs differ even if "cat" and "sat" had identical embeddings.

Two approaches:

Learned:

- Each position 0...N-1 has a trainable embedding vector
- Flexible, but limited to the maximum sequence length seen during training
- Used in BERT, GPT

Sinusoidal (Vaswani et al., 2017):

- $p(i, 2k) = \sin(i / 10000^{(2k/d)})$
- $p(i, 2k+1) = \cos(i / 10000^{(2k/d)})$
- Generalizes to sequences longer than those seen during training

Part 4 — Multi-Head Attention

Q, K, V projections — making attention trainable

$$\text{score}(i, j) = \text{softmax}\left(\frac{(A_Q \mathbf{w}_i) \cdot (A_K \mathbf{w}_j)}{\sqrt{d_k}}\right) \quad \text{output}_i = \sum_j \text{score}(i, j) \cdot A_V \mathbf{w}_j$$

Why not use $\mathbf{W} \cdot \mathbf{W}^T$ directly?

If Query and Key are the same vector $\mathbf{w}_i$, the model can only learn similarity between the original embeddings — with no flexibility.

With separate A_Q , A_K , A_V :

- A_Q transforms $\mathbf{w}_i$ into a "query" (what am I looking for?)
- A_K transforms $\mathbf{w}_j$ into a "key" (what do I have to offer?)
- A_V transforms $\mathbf{w}_j$ into a "value" (what should I contribute?)

Q, K, V names come from retrieval systems:

Query (Q): the question I'm asking

Key (K): the index of what is available

Value (V): the associated content

Attention finds which Keys are most similar to the Query and returns a weighted combination of the corresponding Values.

The matrices A_Q , A_K , A_V are learned via backprop — the model decides what is "relevant" for each task.

Multi-Head Attention

Problem with a single head:

A single attention head learns one similarity pattern. But words relate to each other in multiple ways simultaneously.

Head 1 — Verb tense agreement

"ran" → connects with "yesterday", "quickly"

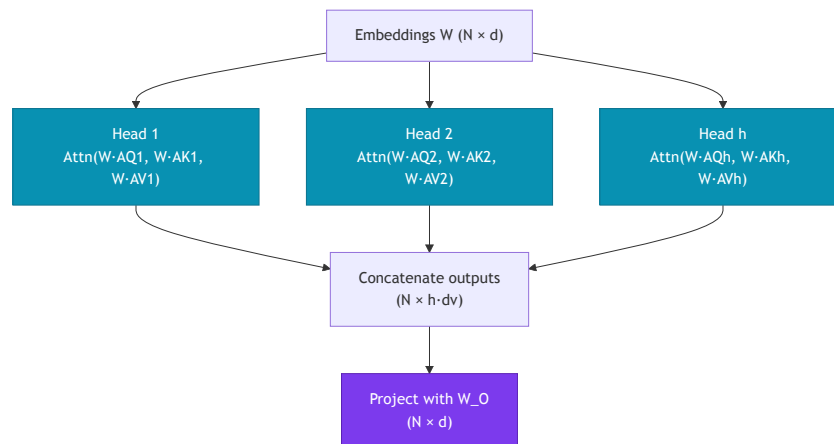
Head 2 — Entity relationships

"president" → connects with the governed country

Head 3 — Tone and polarity

"brilliant" → connects with other positive adjectives

Multi-Head Architecture



Each head has its own learnable matrices A_Q, A_K, A_V, capturing an independent attention pattern. The projection W_O combines all heads

Part 5 — Full Encoder

Residual Connections

Problem: deep Transformers (many layers) suffer from gradients that diminish as they pass through each non-linear transformation.

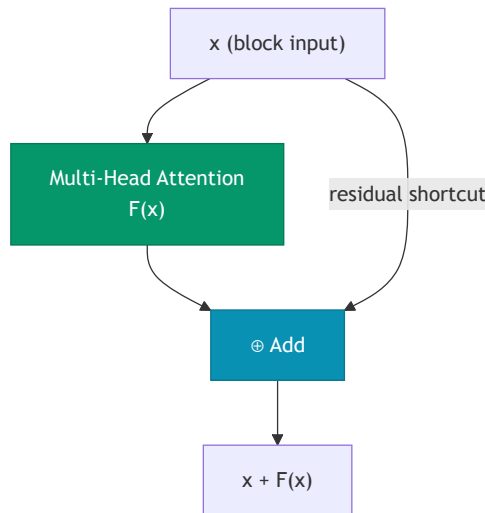
Solution: add the block's input to its output.

Block output = $F(x) + x$

where $F(x)$ is the learned transformation (attention or FFN).

Why does it work?

- The gradient can flow directly through the additive connection
- The network only learns the **difference** (residual) from the input
- Enables training very deep networks (ResNets use the same principle)



In practice: the residual connection is applied both after attention and after the FFN. This allows stacking dozens of layers without gradient degradation.

Layer Normalization

Problem: after linear and non-linear transformations, embedding vectors can have highly variable scale and mean — making training unstable.

Layer Normalization: normalize each embedding individually.

For a vector h of dimension d :

$$\text{mean} = (1/d) \times \sum(h_i)$$

$$\text{std} = \sqrt{(1/d) \times \sum((h_i - \text{mean})^2)}$$

$$h_{\text{norm}} = (h - \text{mean}) / \text{std}$$

$$h_{\text{out}} = \gamma \times h_{\text{norm}} + \beta$$

LayerNorm vs BatchNorm

BatchNorm Normalizes along the batch dimension — problematic for variable-length sequences

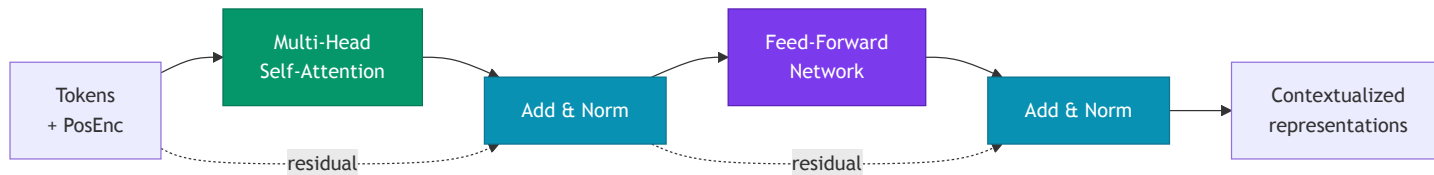
LayerNorm Normalizes along the embedding dimensions — independent of batch size or sequence length

gamma and beta are learnable parameters (scale and shift) — allow the model to recover the original scale if needed.

Result: weights remain stable during training, accelerating convergence even with larger learning rates.

Full Encoder Block

Each block: **MHA** → **Add & Norm** → **FFN** → **Add & Norm** (N times; BERT-base: N = 12)



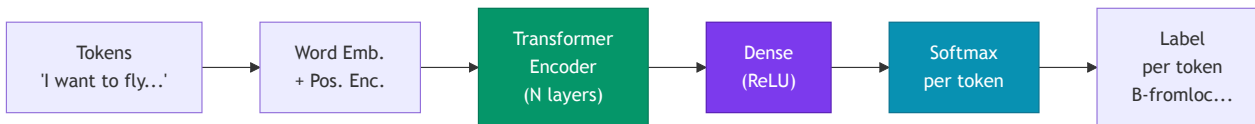
N stacked layers: each block MHA → Add&Norm → FFN → Add&Norm is repeated N times (e.g., BERT-base: N=12).

Optimized weights: positional embeddings, word embeddings, A_Q/A_K/A_V per head, FFN weights, gamma/beta of LayerNorm.

Part 6 — Applications

Slot Filling with Transformer

Complete pipeline for token-by-token classification:



Input Each sentence token is converted to an embedding and added to the corresponding positional encoding.

Encoder The Transformer produces a contextual representation for each token — each vector has "seen" all other tokens.

Classification A dense layer + per-token softmax produces a probability distribution over labels for each position.

Task Types with Transformers

1. Sequence Classification

Use the `[CLS]` token or mean of final embeddings.

```
CLS] sentence... [SEP
```

↓

```
Dense → Softmax
```

↓

```
sentence label
```

Ex: sentiment analysis, spam detection, topic classification.

2. Sequence Labeling

Use each token's embedding individually.

```
tok1 tok2 tok3
```

↓ ↓ ↓

```
Dense → Softmax × N
```

↓ ↓ ↓

```
lab1 lab2 lab3
```

Ex: NER, slot filling, POS tagging.

3. Sequence Generation

Use Transformer decoder with causal attention (each token can only see previous tokens).

```
[START] → tok1 → tok2 → tok3
```

Ex: machine translation, summarization, chatbots.

`[CLS]` (*Classification*) — special token inserted at the start of every input sequence. After the encoder, its embedding aggregates

`[SEP]` (*Separator*) — special token marking the end of a sentence or separating two sentences in pair tasks (e.g., textual entailment). Signals to the model where a sequence ends.

Text Generation: Temperature and Top-k

In **decoder** models, the softmax produces a **distribution over the vocabulary** at each step. How to pick the next token?

Greedy ($\tau \rightarrow 0$)

Always picks the most probable token. Deterministic and fast, but tends to repeat itself.

Temperature (τ)

$\text{softmax}(\text{logits} / \tau)$ — $\tau < 1 \rightarrow$ more focused; $\tau > 1 \rightarrow$ more creative/random; $\tau = 1 \rightarrow$ model's original distribution.

Top-k sampling

Keeps only the k most probable tokens before softmax. $k = 1 =$ greedy; $k = 50$ is common in practice.

Top-p (nucleus sampling)

Keeps the smallest set with cumulative probability $\geq p$ — k varies: smaller when the model is confident.

In practice (LLM APIs): temperature=0.7, top-p=0.9 are typical defaults. Higher values $\rightarrow$ more varied and creative; lower values $\rightarrow$ more deterministic and factual.

PyTorch Code — Transformer Encoder

```
import torch
import torch.nn as nn

# One encoder block: Multi-Head Attention + FFN + Add&Norm
encoder_layer = nn.TransformerEncoderLayer(
    d_model=512,          # embedding dimension
    nhead=8,              # number of attention heads
    dim_feedforward=2048, # internal FFN dimension
    dropout=0.1,
    batch_first=True      # format (batch, seq, d_model)
)

# Stack N encoder blocks
encoder = nn.TransformerEncoder(
    encoder_layer=encoder_layer,
    num_layers=6          # N = 6 blocks (as in the original paper)
)

# Input embeddings: words + position
embed = nn.Embedding(vocab_size, 512)
pos_enc = nn.Embedding(max_seq_len, 512) # learned positional

# Forward pass
x = embed(tokens) + pos_enc(positions) # (batch, seq, 512)
```

Summary

Motivation

- Polysemy requires contextual representations
- Slot filling: each token's label depends on context
- 3 requirements: same length, context, order

Self-Attention

- Representation = weighted average over embeddings
- Weight proportional to dot product (similarity)
- Softmax normalizes; divide by $\sqrt{d}$ stabilizes

Multi-Head

- Multiple heads capture different patterns
- Each head: independent A_Q , A_K , A_V
- Concatenate and project with W_O

Positional Encoding

- Without encoding: attention is order-invariant
- Add positional embedding to word embedding
- Learned (BERT) or sinusoidal (original paper)

Full Encoder

- Q/K/V make attention tunable via backprop
- Residual: gradients flow without attenuation
- LayerNorm: stable weights; N stacked blocks

Next lecture

- BERT: pre-training and fine-tuning
- GPT: autoregressive generation
- Practical applications: QA, summarization

Next lecture

Lecture 7 — BERT and Large Language Models

- Pre-training: Masked LM and Next Sentence Prediction
- Fine-tuning for downstream tasks
- GPT and autoregressive generation
- Comparison BERT vs GPT vs T5
- ChatGPT and instruction-tuned models

For this week

- Notebook `lec06-transformers.ipynb` :
 - Implement self-attention from scratch in NumPy
 - Use `nn.TransformerEncoder` for slot filling
 - Visualize attention weights across different heads
 - Compare Transformer vs bidirectional LSTM on ATIS

Thank you! Questions?

CCM-109 · Deep Learning · UFABC